Implement Dig Dug Lab Tasks

10/17/2026 09:41:50
https://chatgpt.com/c/6ad2f214-b124-83e8-8d2b-3f387f677327

Lab_4_VibeCoding_Student_handout.pdf This is a lab project: Subject ->...

Thinking ● Searched 1 website ✓ Worked for 9s Your Software...

should we add the contents to the same repo or new one?

Use your personal Lab repository, and keep the assigned game reposito...

okay give me full process as i asked in prompt 1

Thinking ● Searched 4 websites ✓ Worked for 12s Software Engineerin...

Lab_4_VibeCoding_Student_ha...

This is a lab project: Subject -> Software engg
Upload the following to your Lab repo under Lab-4 folder,\
a. Videos (before and after)\
b. Updated Code\
c. Chat history exported as a doc/pdf
Refer worksheet for the code assigned to you\
[https://github.com/SETAPESU26/29_digDug]
(https://github.com/SETAPESU26/29_digDug)
Refer to this handout and find solns to the loopholes in the game:
Each task must be completed using an iterative process involving LLM suggestions and your critical code review.\
Task 1: Fix the enemy pathfinding bug\
Once a tunnel connects an enemy to the player, the enemy is supposed to take the shortest route through cleared cells, using breadth-first search. In the current build, enemies do eventually reach the player, but along long, winding routes instead of the shortest one — the search behaves like depth-first search rather than breadth-first. Look at which end of the `deque` the search removes cells from in `bfs_path`.\
Task 2: Implement `dirt_color(row)`\
Called once per dirt cell per frame in `draw`, as `shade = dirt_color(row) or (170 - row * 6, 110 - row * 4, 60)`. It receives the cell's row index and should return an `(r, g, b)` color, or `None` to keep the default gradient (which darkens with depth). Idea: give each row band a distinct color instead of a smooth gradient.\
Task 3: Implement `on_enemy_popped(enemy, score)`\
Called from `pump()` right after an enemy reaches its fourth inflation stage, is removed from play, and its points are added. It receives the `Enemy` that popped and the score after its points were added. Its return value is ignored. Idea: a particle burst at `enemy.cell`,

or extra points for popping an enemy deep underground.\
Task 4: Implement `enemy_speed_multiplier(level)`\
Called every time a non-locked enemy finishes a step, as part of `enemy.step_timer = ENEMY_DELAY * (1.6 if enemy.ghost else 1) / (enemy_speed_multiplier(self.level) or 1)`. It receives the current level number and should return a speed multiplier, or `None` for the default speed (1x). A higher multiplier makes enemies move more often (shorter delay between steps). Idea: return `1 + 0.1 * level` so enemies get gradually faster on deeper levels.

17/10/2026, 09:27:15

Thinking

- **Searched 1 website**

✓ Worked for 9s

Your Software Engineering Lab-4 is an individual **Vibe Coding assignment**. You need to fix one existing bug and implement three features in the Dig Dug game, using an iterative process of LLM suggestions and your own code review.

The handout also requires separate commits for each task, a 10-second video before changes, a 10-second video after changes, and the final deliverables inside your Lab repository's `Lab-4` folder. Do not raise a pull request to the original repository.

1. Solutions to the four tasks

Task 1 — Fix BFS enemy pathfinding

Problem: The search removes the newest cell from the `deque`, causing depth-first-style exploration instead of breadth-first search.

Fix: In `bfs_path()` in `game.py`, change the queue removal to FIFO order.

```
# Incorrect: removes the newest cell (LIFO)
current = queue.pop()
```

```
# Correct: removes the oldest cell (FIFO)
current = queue.popleft()
```

Keep the existing neighbor exploration and path reconstruction logic unchanged.

Why this works: BFS explores cells level by level. In a grid where each move has equal cost, this finds the shortest path through cleared tunnels.

Task 2 — Implement `dirt_color(row)`

Give different underground row bands distinct colors. This version preserves the default gradient for rows outside the explicitly defined bands.

```
def dirt_color(row):
    if row < 3:
        return (194, 156, 92)
    elif row < 6:
        return (174, 126, 75)
    elif row < 9:
        return (145, 95, 61)
    else:
        return (105, 65, 48)
```

Review: The game calls this function once per dirt cell per frame, so it should be fast and should not modify game state. Each returned tuple is an RGB color. Adjust the row thresholds if needed to match the game's grid dimensions.

Task 3 — Implement `on_enemy_popped(enemy, score)`

Add a bonus for defeating enemies deeper underground. The function receives the enemy and the score after the normal points have already been awarded.

```
def on_enemy_popped(enemy, score):
    row = enemy.cell[1]
    bonus = max(0, row - 2) * 10
    return bonus
```

Important review finding: The return value is ignored by the existing game, so this function alone will calculate a bonus but will **not add it to the score**.

For an actual scoring feature, modify the existing `pump()` call site to apply the bonus:

```
bonus = on_enemy_popped(enemy, self.score)
self.score += bonus
```

Use the actual score variable and attribute names in `game.py`. Check whether the enemy's cell is represented as `(column, row)` before using index `1`. This implementation gives 10 extra points per row below row 2.

Task 4 — Implement `enemy_speed_multiplier(level)`

Make enemies gradually faster as the level increases.

```
def enemy_speed_multiplier(level):
    return 1 + 0.1 * level
```

Examples:

Level	Multiplier	Relative delay
1	1.1×	$1/1.1$ of normal
3	1.3×	$1/1.3$ of normal
5	1.5×	$1/1.5$ of normal

A higher multiplier shortens the delay between enemy steps. This follows the handout's suggested formula. [GitHub - SETAPESU26/29_digDug: Implementation of Dig Dug in Pygame.]

Recommended final implementation

The repository confirms that `enemy.cell` stores `(row, column)`, the score is `self.score`, and `pump()` already calls `on_enemy_popped(enemy, self.score)` after awarding the normal points. [29_digDug/game.py at main · SETAPESU26/29_digDug]

Replace the three empty functions and fix the queue operation in `game.py` with this implementation:

```
def dirt_color(row):
    if row < 3:
        return (194, 156, 92)
    elif row < 6:
        return (174, 126, 75)
    elif row < 9:
        return (145, 95, 61)
    return (105, 65, 48)


def on_enemy_popped(enemy, score):
    row = enemy.cell[0]
    bonus = max(0, row - 2) * 10
    return bonus


def enemy_speed_multiplier(level):
    return 1 + 0.1 * level
```

In `bfs_path()`, change:

```
cell = queue.pop()
```

to:

```
cell = queue.popleft()
```

Finally, update the existing section in `pump()`:

```
self.score += 200 + 100 * (enemy.cell[0] // 4)

self.pump_target = None

bonus = on_enemy_popped(enemy, self.score)
self.score += bonus
```

This makes the deep-underground bonus actually affect the score. The `score` parameter represents the score before this bonus is added.

2. Complete the lab using iterative LLM review

The handout explicitly requires you to review and test the LLM's suggestions rather than blindly accepting them.

Use a separate commit for each task.

Step 1 — Record the original game

Before editing, run the original game and record a 10-second video showing enemy movement. Save it as `before.mp4`.

Step 2 — Fix BFS and commit

Ask the LLM to identify the queue behavior, review the suggested `popleft()` change, test the pathfinding, and commit the change.

Step 3 — Add dirt colors and commit

Implement `dirt_color(row)`, check that different depth bands look distinct, and verify that the default gradient remains available when the function returns `None`.

Step 4 — Add the popping bonus and commit

Implement `on_enemy_popped()`, connect its returned bonus to `self.score`, and check that the score increases correctly when an enemy pops.

Step 5 — Increase enemy speed and commit

Implement the multiplier, verify that enemies move faster at higher levels, and check that ghost movement and locked enemies retain their intended behavior.

Step 6 — Final testing and recording

Play the updated game, verify all four changes, and record a 10-second `after.mp4` video.

3. Terminal commands on your Mac

Clone the assigned repository, then create your own repository under your personal GitHub account. Work in your personal repository, not the original assignment repository.

```
git clone https://github.com/SETAPESU26/29_digDug.git
cd 29_digDug

python3 -m pip install pygame
python3 game.py
```

Record the original game before modifying `game.py`.

Once you have created your own repository, add it as the remote for your working copy. Replace the placeholder with your actual GitHub username and repository name:

```
git remote rename origin upstream
git remote add origin https://github.com/YOUR_USERNAME/YOUR_LAB_REPO.git

git switch -c lab-4
mkdir -p Lab-4
```

After each task, run the game and commit only that task's change. For example:

```
git add game.py
git commit -m "Fix enemy BFS pathfinding"
```

Use these commit messages for the remaining tasks:

```
git commit -am "Implement depth-based dirt colors"
git commit -am "Add bonus points for popped enemies"
git commit -am "Increase enemy speed by level"
```

The `git commit -am` command stages modifications to already tracked files. If you add new files, use `git add` for those files as well.

After testing, copy the two videos and your exported chat-history PDF into `Lab-4/`. Make sure the updated `game.py` is included there too.

```
git add Lab-4
git commit -m "Add Lab 4 deliverables"
git push -u origin lab-4
```

4. Final submission checklist

Lab-4 submission 0 of 6

- [ ] 10-second original gameplay video
- [ ] 10-second updated gameplay video
- [ ] Updated game.py
- [ ] Complete ChatGPT conversation exported as PDF
- [ ] Four separate commits for the four tasks
- [ ] All deliverables uploaded under Lab-4 in your personal repository

Chat history: Export this conversation using ChatGPT's available sharing or printing options and save it as a PDF. Include the iterative prompts, your code-review observations, the corrections you made, and the tests you performed. Keep the original conversation intact so it accurately documents your work.

One final detail: the proposed dirt colors and depth bonus are implementation choices, not fixed values specified by the handout. Test them in the actual game before submitting.

17/10/2026, 09:27:18 · gpt-6

should we add the contents to the same repo or new one?

17/10/2026, 09:28:04

Use your personal Lab repository, and keep the assigned game repository as the source.

The handout says to clone or fork the assigned repository, make changes, and push the code to your personal repository. It also says not to raise a PR to the original repository.

- **Assigned repo:** github.com
- **Destination:** Your existing personal Lab repository.
- **Folder:** `Lab-4/`

Your final structure should look like this:

```
Your-Lab-Repo/
└── Lab-4/
    ├── game.py
    ├── before.mp4
    ├── after.mp4
    └── chat_history.pdf
```

If your personal Lab repository already exists, **don't create another repository**. Add the `Lab-4` folder to it and push your changes.

One distinction: keep your working game project intact while developing and testing it. Copy the final updated code and deliverables into `Lab-4/` before committing and pushing.

17/10/2026, 09:28:06 · gpt-6

okay give me full process as i asked in prompt 1

17/10/2026, 09:28:58

Thinking

- **Searched 4 websites**

✓ Worked for 12s

Software Engineering Lab-4 — Complete Step-by-Step Process

Your assignment is to fix the Dig Dug game, add three features, review the LLM's suggestions, and submit everything to your **existing personal Lab repository under** `Lab-4/`.

The original assigned repository is github.com.

The handout requires a 10-second video before changes, a 10-second video after changes, updated code, exported chat history, and separate commits for each task. You must work individually and must not raise a PR to the original repository.

We'll follow this order so you don't lose your original version or accidentally push your work to the wrong repository.

Phase 1 — Set up the project

Step 1. Open Terminal and clone the assigned repository

Run these commands one by one on your Mac:

```
cd ~/Downloads
git clone https://github.com/SETAPESU26/29_digDug.git
cd 29_digDug
```

Check the files:

```
ls
```

Read the README before editing:

```
cat README.md
```

If the README is named differently or isn't in the root folder, use `ls` to locate it.

Step 2. Create a baseline copy

Before making changes, preserve the original code:

```
cp game.py game_original.py
```

If `game.py` is not the actual filename, use the filename shown by `ls`.

Check the Python version and install Pygame:

```
python3 --version
python3 -m pip install pygame
```

If Pygame is already installed, you can continue.

Step 3. Run the original game

```
python3 game.py
```

If the README specifies a different launch command, follow that instead.

Confirm that the game starts and that you can see the gameplay. Don't edit anything yet.

Step 4. Record the original gameplay

Use your Mac's screen-recording feature or QuickTime Player to capture approximately 10 seconds of gameplay.

Save the video somewhere convenient, such as your Downloads folder, as:

```
before.mp4
```

Try to capture enemy movement and the existing dirt appearance. This is your evidence of the original build.

Stop the game when you have finished recording.

Phase 2 — Create your personal Lab repository workflow

There are two possible situations. Follow the one that matches your setup.

- **If your personal Lab repository already exists:** use it as the destination. Do not create another repository.
- **If it doesn't exist yet:** create a repository on GitHub using your course's required naming convention, then use its URL below.

Step 5. Create a working branch

From inside the cloned game directory:

```
git switch -c lab-4
git status
```

This creates a separate branch for your lab changes.

Step 6. Connect your personal Lab repository

First, check your existing remote:

```
git remote -v
```

The assigned repository should currently be named `origin`. Rename it to `upstream`, then add your personal Lab repository as `origin`:

```
git remote rename origin upstream
git remote add origin https://github.com/YOUR_USERNAME/YOUR_LAB_REPO.git
```

Replace both placeholders with your actual details.

Verify:

```
git remote -v
```

You should see the assigned repository under `upstream` and your personal Lab repository under `origin`.

Do not push to `upstream`. Your final work must go to your own repository.

Phase 3 — Complete the four tasks iteratively

Open the project in your preferred editor. If you use VS Code:

```
code .
```

Locate `bfs_path`, `dirt_color`, `on_enemy_popped`, `enemy_speed_multiplier`, and the `pump()` method before making changes.

For every task, follow this loop:

1. Ask the LLM for a targeted solution.
2. Review the relevant code yourself.
3. Apply only the necessary change.
4. Run and test the game.
5. Ask the LLM to review any issues you found.
6. Commit the completed task separately.

Task 1 — Fix enemy pathfinding

Prompt to use:

> Act as a Python game developer. Inspect the existing `bfs_path()` implementation. Enemies should find the shortest route through cleared cells using breadth-first search, but the current implementation explores paths in a depth-first manner.
>
> Identify the exact line responsible. Explain why it is wrong, provide the smallest possible fix, and do not rewrite unrelated code. Preserve the existing neighbor handling and path reconstruction.

The intended correction is to remove the oldest queued cell instead of the newest one. In a Python `deque`, that usually means changing `pop()` to `popleft()`.

Your code review: Confirm that the queue is initialized correctly, visited cells are not repeatedly explored, and the reconstructed path leads from the enemy toward the

player.

Test a layout where a longer winding route and a shorter route are both available.

Commit the change:

```
git add game.py
git commit -m "Fix enemy BFS pathfinding"
```

Use the actual filename if it differs from `game.py`.

Task 2 — Implement `dirt_color(row)`

Prompt to use:

> Implement `dirt_color(row)` in the existing game. It receives a dirt cell's row index and must return an RGB tuple or `None`. Use distinct color bands for different depths rather than a smooth gradient. Keep the function deterministic, efficient, and independent of game state.
>
> Explain your chosen row ranges and colors. Preserve the existing fallback expression so that returning `None` continues to use the default gradient. Change no unrelated rendering code.

Your code review:

- Confirm each return value contains three valid RGB integers from 0 to 255.
- Check that rows in different bands produce different colors.
- Check that the fallback gradient still works if `None` is returned.
- Check that the function does not alter the game state.

Commit:

```
git add game.py
git commit -m "Implement depth-based dirt colors"
```

Task 3 — Implement `on_enemy_popped(enemy, score)`

Prompt to use:

Implement `on_enemy_popped(enemy, score)` in the existing game. It is called after an enemy reaches its fourth inflation stage, is removed, and its normal points have been added to the score.

Add a bonus based on how deep underground the enemy is when popped. First inspect how `enemy.cell` stores its coordinates and identify which index represents the row. Explain the bonus formula.

Crucially, inspect the caller in `pump()`: the hook's return value is currently ignored. Make the smallest necessary change so the bonus is actually added to the score exactly once. Do not double-count the normal points or change unrelated gameplay.

Your code review:

- Verify whether the cell uses `(row, column)` or `(column, row)`.
- Confirm the bonus increases with depth and never becomes negative.
- Confirm normal points are still awarded once.
- Pop enemies at different depths and compare the score.

If you use a bonus formula such as `max(0, row - 2) * 10`, check that it behaves correctly for the game's actual row indices.

Commit:

```
git add game.py
git commit -m "Add bonus points for popped enemies"
```

Task 4 — Implement `enemy_speed_multiplier(level)`

Prompt to use:

Implement `enemy_speed_multiplier(level)` so enemies gradually move faster as the level increases. Follow the suggested formula `1 + 0.1 * level`.

Inspect how the multiplier is used in the existing `enemy.step_timer` calculation. Preserve the existing behavior for ghost enemies and locked enemies. Keep the change minimal and explain how the multiplier affects the movement delay.

Your code review:

The suggested formula is:

```
def enemy_speed_multiplier(level):
    return 1 + 0.1 * level
```

Because the multiplier divides the step delay, higher values make the delay shorter.

Check at least two levels and ensure the game remains playable. Also verify that locked enemies do not start moving just because the multiplier is larger.

Commit:

```
git add game.py
git commit -m "Increase enemy speed by level"
```

Phase 4 — Test the final game

After all four tasks are committed, run the game again:

```
python3 game.py
```

Test each feature against this checklist.

- [ ] BFS pathfinding — Enemies take the shortest available route through connected cleared cells.
- [ ] Dirt colors — Different depth bands have distinct colors; the default gradient remains available.
- [ ] Enemy popping bonus — Popping an enemy awards normal points plus the depth-based bonus, exactly once.
- [ ] Level-based speed — Enemies move faster at higher levels without breaking ghost or locked behavior.
- [ ] Regression check — Player movement, pumping, scoring, and level progression still work.

If any test fails, return to the relevant task, correct the implementation, rerun the test, and make a separate corrective commit.

Phase 5 — Record the updated gameplay

Once all tests pass:

1. Launch the updated game.
2. Record another 10-second gameplay video.
3. Show the changed appearance and gameplay wherever possible.
4. Save it as `after.mp4`.

Keep `before.mp4` unchanged so you can demonstrate the difference between the original and updated game.

Phase 6 — Export your LLM chat history

You need a document or PDF containing the actual conversation used during the assignment.

1. Keep your task-specific prompts and the LLM's responses.
2. Include your code-review questions and any follow-up corrections.
3. Include the final code changes and the testing observations.
4. Export the conversation using the available ChatGPT sharing, printing, or export options.
5. Save the resulting PDF as `chat_history.pdf`.

Do not fabricate prompts or claim to have run tests that you did not perform. Include the real interaction history from your work.

Phase 7 — Put the deliverables under `Lab-4/`

Your existing personal Lab repository should ultimately contain:

```
Your-Lab-Repo/
├── ... existing lab folders ...
└── Lab-4/
    ├── game.py
    ├── before.mp4
    ├── after.mp4
    └── chat_history.pdf
```

If the game requires other Python modules, images, sound files, or configuration files to run, include the necessary project files too. Don't submit a standalone `game.py` that cannot run without its dependencies.

From your game project's root directory, create the submission folder if necessary:

```
mkdir -p Lab-4
```

Copy the final code into it:

```
cp game.py Lab-4/game.py
```

Copy your recordings and exported chat history from wherever you saved them. For example, if all three files are in Downloads:

```
cp ~/Downloads/before.mp4 Lab-4/
cp ~/Downloads/after.mp4 Lab-4/
cp ~/Downloads/chat_history.pdf Lab-4/
```

If your personal Lab repository is a separate directory, copy the entire completed `Lab-4` folder into that repository instead.

Important: If the game needs additional files, preserve the directory structure and include those dependencies in `Lab-4/` as well.

Phase 8 — Push the deliverables

If you're already working inside your personal Lab repository, run:

```
git status
git add Lab-4/
git commit -m "Add Lab 4 deliverables"
git push -u origin lab-4
```

If your course expects submissions on the `main` branch, push to the required branch instead. Do not force-push or overwrite existing coursework.

Verify that all files appear in the `Lab-4` folder on your personal GitHub repository.

Final submission checklist

- [ ] Original gameplay video, 10 seconds

- [ ] Updated gameplay video, 10 seconds
- [ ] Updated code and any required dependencies
- [ ] Complete LLM chat history exported as PDF
- [ ] Four separate commits for the four assigned tasks
- [ ] All deliverables uploaded under `Lab-4/`
- [ ] No pull request raised to the original repository

One last precaution: **inspect your actual `game.py` before applying the suggested code.** The task descriptions establish the intended behavior, but the precise coordinate indexing, score variable, launch command, and file layout must match your assigned checkout. Review those details before committing.

17/10/2026, 09:29:03 · gpt-6